



UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR

Master 2 Business Intelligence — Cours de Traitement Automatique du Langage Naturel

PROJET DE FIN DE COURS

Répondre à des questions de santé en akan, luganda, swahili et amharique

Challenge Zindi — Modèles de questions-réponses médicales multilingues

Comparaison de trois approches : retrieval TF-IDF, LLM multilingue NLLB-200 (zero-shot et fine-tuné), mT5-small (full fine-tuning) et Gemma E2B (QLoRA)

PRÉSENTÉ PAR

Andil Mchangama

Nasma Youssouf M'madi

Abdoulaye Diouf

Harouna Mbaye

Hamidou Mbodji

Dakar, 2026

SOMMAIRE

1. Contexte et objectifs du projet
2. Point de départ du challenge — baselines fournies par Zindi
3. Exploration des données
4. Modèle 1 — mT5-small (full fine-tuning)
5. Modèle 2 — Gemma E2B (QLoRA)
6. Comparaison des modèles
7. Difficultés techniques rencontrées et résolues
8. Conclusion et recommandations

1. Contexte et objectifs du projet

Le challenge vise à entraîner un modèle capable de répondre à des questions de santé — santé sexuelle et reproductive notamment — directement dans la langue locale de l'utilisateur, plutôt qu'en anglais uniquement. C'est ce choix de terrain qui a guidé nos décisions tout au long du projet : une bonne partie des populations concernées ne lit ni n'écrit l'anglais couramment, et une réponse médicale mal comprise perd toute son utilité, aussi juste soit-elle sur le plan clinique.

Langues couvertes par le jeu de données, réparties en 8 sous-ensembles pays/langue.

Langue	Pays / subset
English	Uganda (Eng_Uga), Ghana (Eng_Gha), Éthiopie (Eng_Eth), Kenya (Eng_Ken)
Akan	Ghana (Aka_Gha)
Luganda	Uganda (Lug_Uga)
Swahili	Kenya (Swa_Ken)
Amharique	Éthiopie (Amh_Eth)

Pour répondre à ce défi, nous avons choisi d'explorer **deux approches complémentaires** plutôt que de nous limiter à une seule, afin de pouvoir comparer leurs compromis respectifs :

- **mT5-small** (Google) — un modèle Sequence-to-Sequence multilingue, que nous avons entraîné en *full fine-tuning* (tous les paramètres mis à jour) ;
- **Gemma E2B** (Google) — un modèle génératif plus récent et beaucoup plus volumineux, que nous avons adapté par *QLoRA* (quantification 4-bit + adaptateurs LoRA), une technique d'adaptation efficace en paramètres.

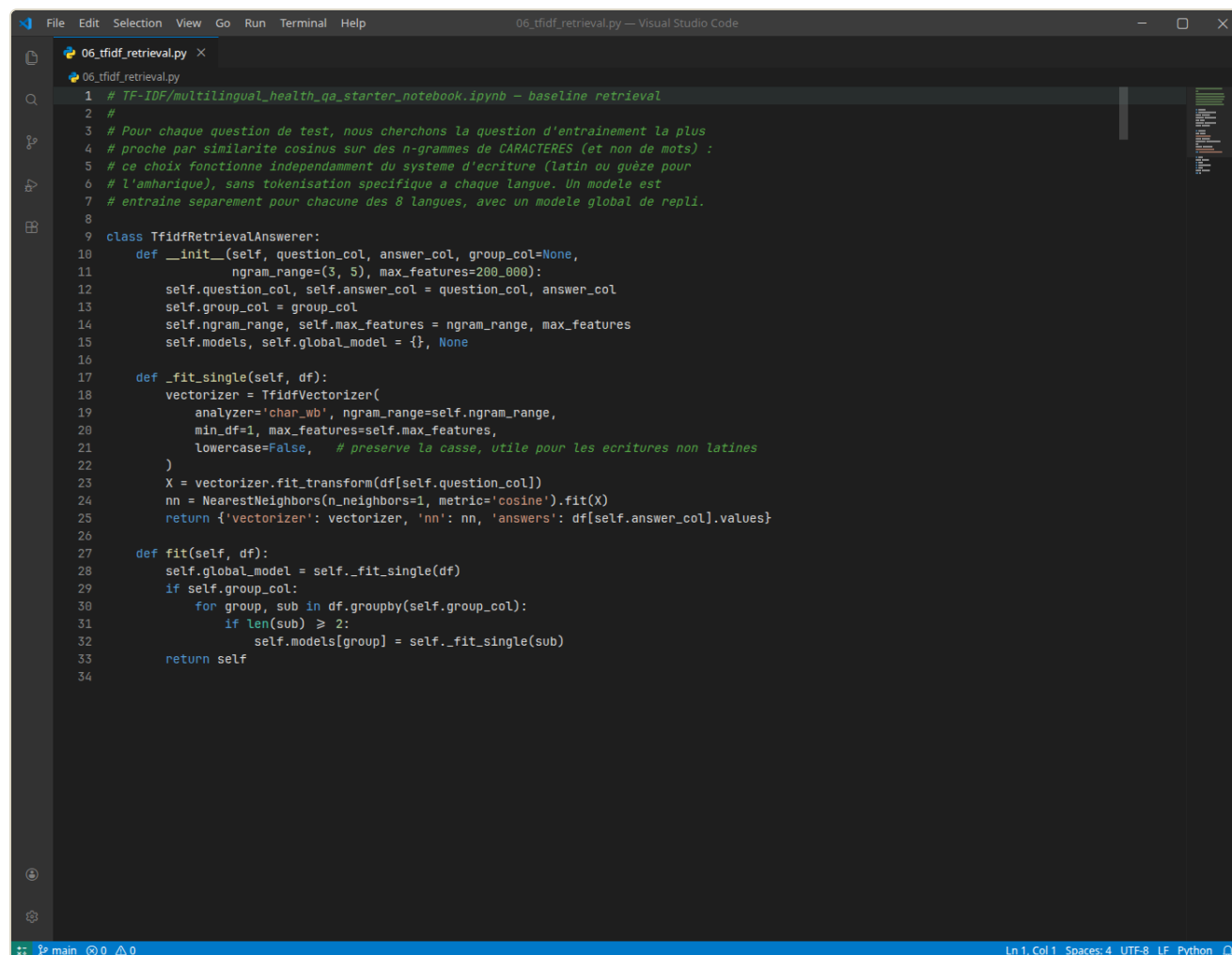
Ce choix n'était pas arbitraire : mT5-small est le modèle explicitement recommandé par le notebook de démarrage Zindi pour un premier run rapide (section suivante), tandis que Gemma E2B nous permettait de tester une approche plus ambitieuse, à la mesure de ce qu'un modèle génératif récent peut apporter sur cette tâche — au prix d'un budget de calcul largement supérieur, que nous détaillons plus loin.

2. Point de départ du challenge — baselines fournies par Zindi

Avant de développer nos propres pipelines, nous avons pris connaissance du notebook de démarrage (« *starter notebook* ») fourni par Zindi. Ce notebook n'est pas notre travail : c'est le code de référence proposé par l'organisateur, que nous avons exécuté tel quel pour disposer d'un point de comparaison indépendant avant d'entraîner nos deux modèles.

2.1 Baseline 1 — Retrieval TF-IDF

La première approche ne fait appel à aucun réseau de neurones : pour chaque question de test, elle recherche la question d'entraînement la plus proche par similarité cosinus sur des **n-grammes de caractères** — un choix qui fonctionne indépendamment du système d'écriture (latin ou guèze pour l'amharique), sans avoir besoin d'un tokeniseur propre à chaque langue.



```
1 # TF-IDF/multilingual_health_qa_starter_notebook.ipynb - baseline retrieval
2 #
3 # Pour chaque question de test, nous cherchons la question d'entraînement la plus
4 # proche par similarité cosinus sur des n-grammes de CARACTÈRES (et non de mots) :
5 # ce choix fonctionne indépendamment du système d'écriture (latin ou guèze pour
6 # l'amharique), sans tokenisation spécifique à chaque langue. Un modèle est
7 # entraîné séparément pour chacune des 8 langues, avec un modèle global de repli.
8
9 class TfidfRetrievalAnswerer:
10     def __init__(self, question_col, answer_col, group_col=None,
11                  ngram_range=(3, 5), max_features=200_000):
12         self.question_col, self.answer_col = question_col, answer_col
13         self.group_col = group_col
14         self.ngram_range, self.max_features = ngram_range, max_features
15         self.models, self.global_model = {}, None
16
17     def _fit_single(self, df):
18         vectorizer = TfidfVectorizer(
19             analyzer='char_wb', ngram_range=self.ngram_range,
20             min_df=1, max_features=self.max_features,
21             lowercase=False, # preserve la casse, utile pour les écritures non latines
22         )
23         X = vectorizer.fit_transform(df[self.question_col])
24         nn = NearestNeighbors(n_neighbors=1, metric='cosine').fit(X)
25         return {'vectorizer': vectorizer, 'nn': nn, 'answers': df[self.answer_col].values}
26
27     def fit(self, df):
28         self.global_model = self._fit_single(df)
29         if self.group_col:
30             for group, sub in df.groupby(self.group_col):
31                 if len(sub) >= 2:
32                     self.models[group] = self._fit_single(sub)
33         return self
34
```

Extrait — `TfidfRetrievalAnswerer`, baseline de retrieval par n-grammes de caractères.

Langue	ROUGE-1	ROUGE-L
Swa_Ken	0,6031	0,5672
Eng_Ken	0,5989	0,5606
Eng_Eth	0,5170	0,4994
Eng_Uga	0,5163	0,4709
Lug_Uga	0,5155	0,4935
Aka_Gha	0,2832	0,1674
Eng_Gha	0,2582	0,1707
Amh_Eth	0,1455	0,1353

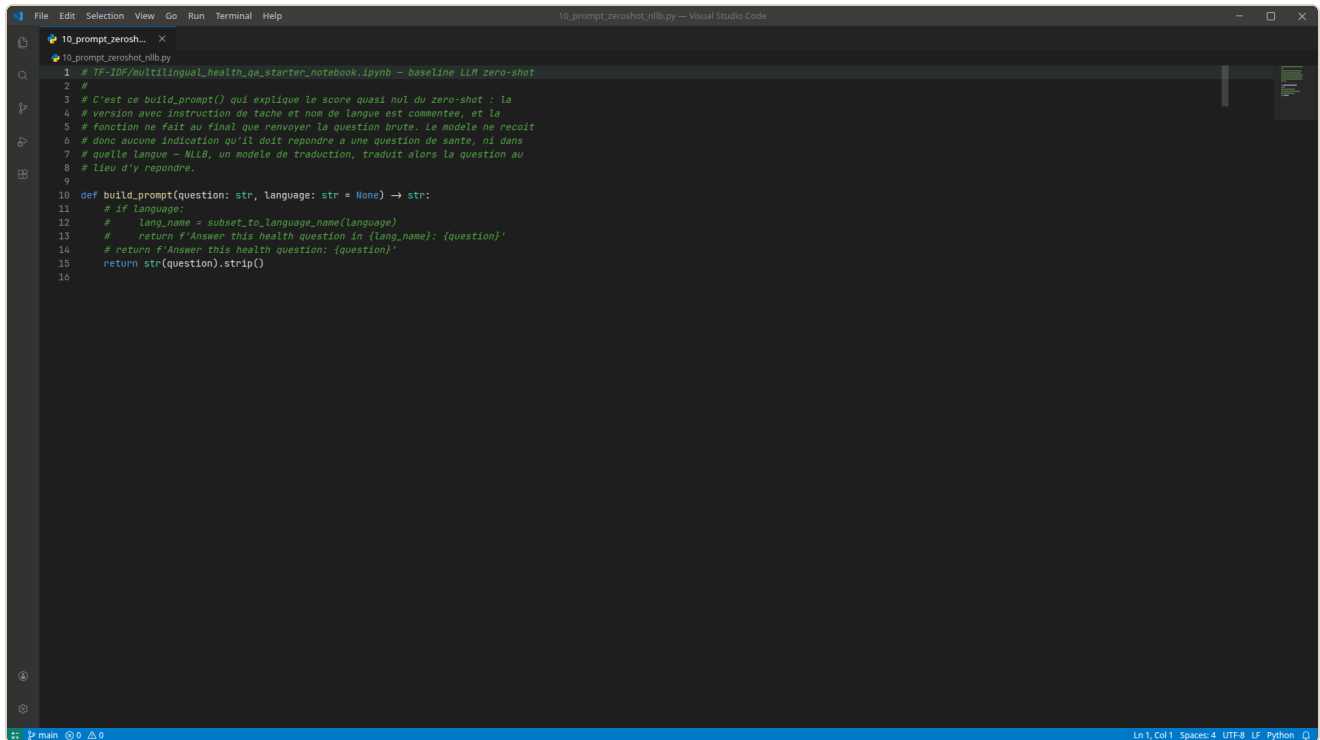
Sur l'ensemble du jeu de validation : ROUGE-1 = 0,4207 / ROUGE-L = 0,3655.



Constat — Ce résultat nous a surpris par sa solidité pour une méthode aussi simple, sans aucun entraînement de réseau de neurones — nous l'avons gardé comme repère de comparaison pour la suite. Nous restons toutefois prudents sur ce que ce score signifie réellement : le TF-IDF *recopie* une réponse humaine existante plutôt que d'en générer une nouvelle, ce qui gonfle mécaniquement son score ROUGE sans garantir que la réponse recopiée réponde bien à la question posée.

2.2 Baseline 2 — LLM multilingue en zero-shot

Le notebook propose trois modèles possibles en commentaire (`google/mt5-small`, `google/mt5-base`, `facebook/nllb-200-distilled-600M`) — seul ce dernier est effectivement chargé dans le run que nous avons exécuté. Le prompt utilisé ne contient aucune instruction de tâche ni nom de langue : la question brute est envoyée telle quelle au modèle.



```
1 # TF-IDF/multilingual_health_qa_starter_notebook.ipynb - baseline LLM zero-shot
2 #
3 # C'est ce build_prompt() qui explique le score quasi nul du zero-shot : la
4 # version avec instruction de tâche et nom de langue est commentée, et la
5 # fonction ne fait au final que renvoyer la question brute. Le Modèle ne reçoit
6 # donc aucune indication qu'il doit répondre à une question de santé, ni dans
7 # quelle langue - NLLB, un modèle de traduction, traduit alors la question au
8 # lieu d'y répondre.
9
10 def build_prompt(question: str, language: str = None) -> str:
11     # if language:
12     #     lang_name = subset_to_language_name(language)
13     #     return f'Answer this health question in {lang_name}: {question}'
14     # return f'Answer this health question: {question}'
15     return str(question).strip()
16
```

Extrait — `build_prompt()`, la version avec instruction de tâche est commentée dans le notebook de départ.

Sur un échantillon de 200 exemples de `Val.csv` (le notebook de démarrage n'évalue pas cette baseline sur l'ensemble complet, contrairement au retrieval TF-IDF ci-dessus) : ROUGE-1 = 0,0074 / ROUGE-L = 0,0071, avec plusieurs langues exactement à 0. En inspectant les réponses générées, nous avons compris pourquoi : NLLB est un modèle de **traduction**, pas de question-réponse, et sans instruction de tâche, il traduit la question au lieu d'y répondre — un exemple généré finit en russe, un autre en français, sans lien avec une réponse médicale.

2.3 Comparaison des deux baselines

Baseline	ROUGE-1 F1	ROUGE-L F1
TF-IDF Retrieval	0,4135	0,3581
LLM zero-shot (NLLB-200)	0,0074	0,0071

2.4 Fine-tuning bonus du LLM (NLLB)

Le notebook propose enfin de fine-tuner ce même NLLB sur nos données d'entraînement (3 epochs, batch 8, Seq2SeqTrainer).

```
11_finetune_config.py
1 # TF-IDF/multilingual_health_qa_starter_notebook.ipynb - Fine-tuning bonus NLI
2 #
3 # Configuration fournie par le notebook de démarrage pour le fine-tuning
4 # bonus : Seq2SeqTrainer standard, 3 epochs, batch 8, sur l'ensemble du jeu
5 # d'entraînement (toutes langues confondues, sans rééquilibrage).
6
7 FINETUNE_OUTPUT_DIR = './mt5-finetuned-health-qa'
8 FINETUNE_EPOCHS = 3
9 FINETUNE_BATCH_SIZE = 8 # Reduce to 4 if you hit OOM errors
10 FINETUNE_LEARNING_RATE = 5e-5
11 FINETUNE_MAX_INPUT_LEN = 256 # Must match MAX_INPUT_LENGTH used at inference
12 FINETUNE_MAX_TARGET_LEN = 512 # Must match MAX_OUTPUT_LENGTH used at inference
13 FINETUNE_VAL_SIZE = 0.05 # 5% of training data used for validation
14
```

Extrait — configuration du fine-tuning bonus proposée par le notebook de démarrage.

Résultats par langue (jeu de validation complet, fine-tuné).

Langue	ROUGE-1	ROUGE-L
Eng_Uga	0,4122	0,3527
Eng_Eth	0,3930	0,3683
Aka_Gha	0,3481	0,2243
Eng_Gha	0,3302	0,2457
Swa_Ken	0,2917	0,2013
Eng_Ken	0,2534	0,1777
Lug_Uga	0,1668	0,1243
Amh_Eth	0,1356	0,1248

Sur l'ensemble du jeu de validation (6 686 exemples) : ROUGE-1 = 0,3176 / ROUGE-L = 0,2484.



Constat — Le fine-tuning fait un bond considérable par rapport au zero-shot (de $\sim 0,007$ à $\sim 0,32$), mais reste en dessous du simple retrieval TF-IDF ($0,41/0,36$) — un rappel utile qu'un plus gros modèle fine-tuné ne bat pas toujours une méthode naïve bien conçue, surtout sans prompt informatif sur la langue cible. Nous retrouvons déjà ici, pour une troisième méthode indépendante, le même constat que dans nos deux pipelines : **l'amharique et le luganda restent les langues les plus faibles**, ce qui nous a renforcés dans l'idée que ces deux langues sont structurellement les plus difficiles du jeu de données, quel que soit le modèle utilisé.

Le notebook Zindi suggère explicitement **mT5-small comme premier choix**, « pour un premier run rapide », avant de recommander de passer à des modèles plus lourds. C'est directement ce qui a motivé notre choix de l'approfondir en un pipeline complet de full fine-tuning (section 4), en complément d'une seconde approche plus ambitieuse sur Gemma E2B (section 5).

3. Exploration des données

Nos deux pipelines partent du même jeu de données brut. Nous avons mené cette exploration deux fois, indépendamment, dans chacun des deux pipelines — et nous sommes arrivés aux mêmes constats de base, ce qui nous a rassurés sur leur fiabilité.

3.1 Chargement et contrôle qualité

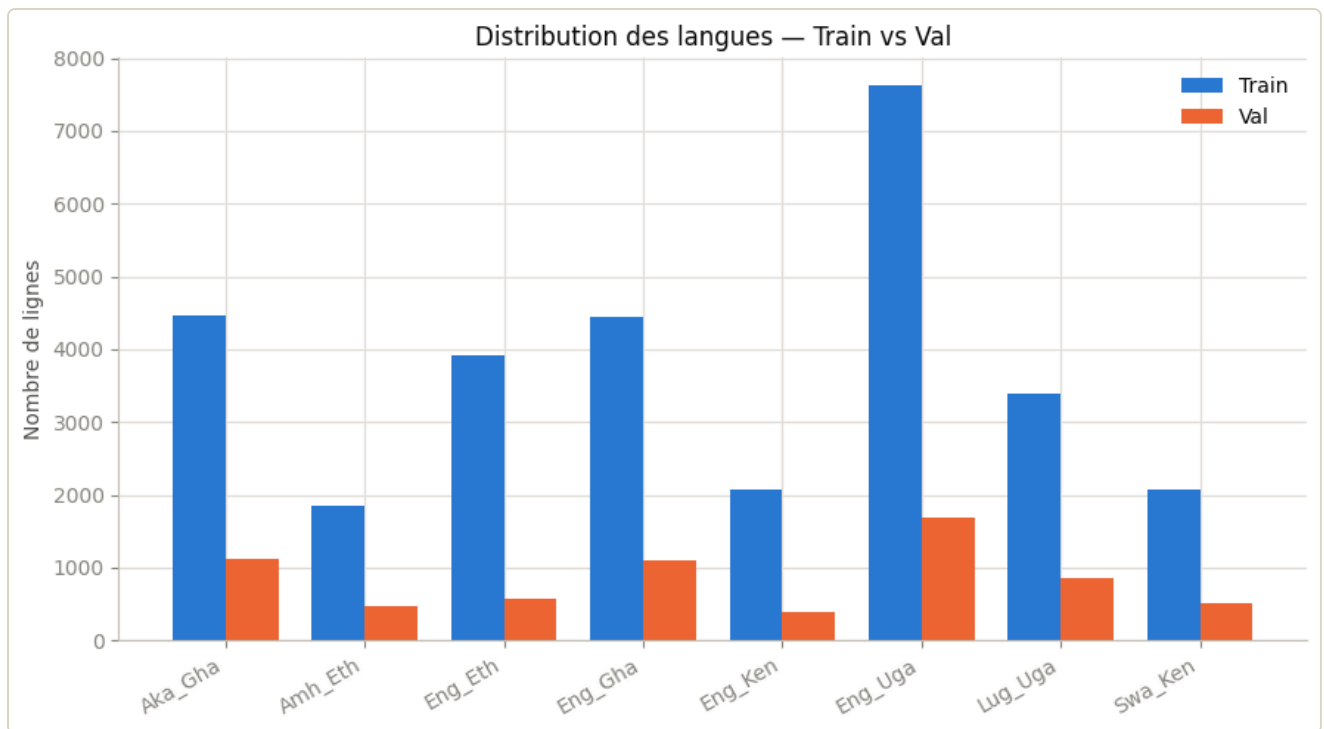
Jeu	Lignes	Colonnes
Train	29 815	4 (ID, input, output, subset)
Val	6 686	4
Test	2 618	3 (output absent, à prédire)



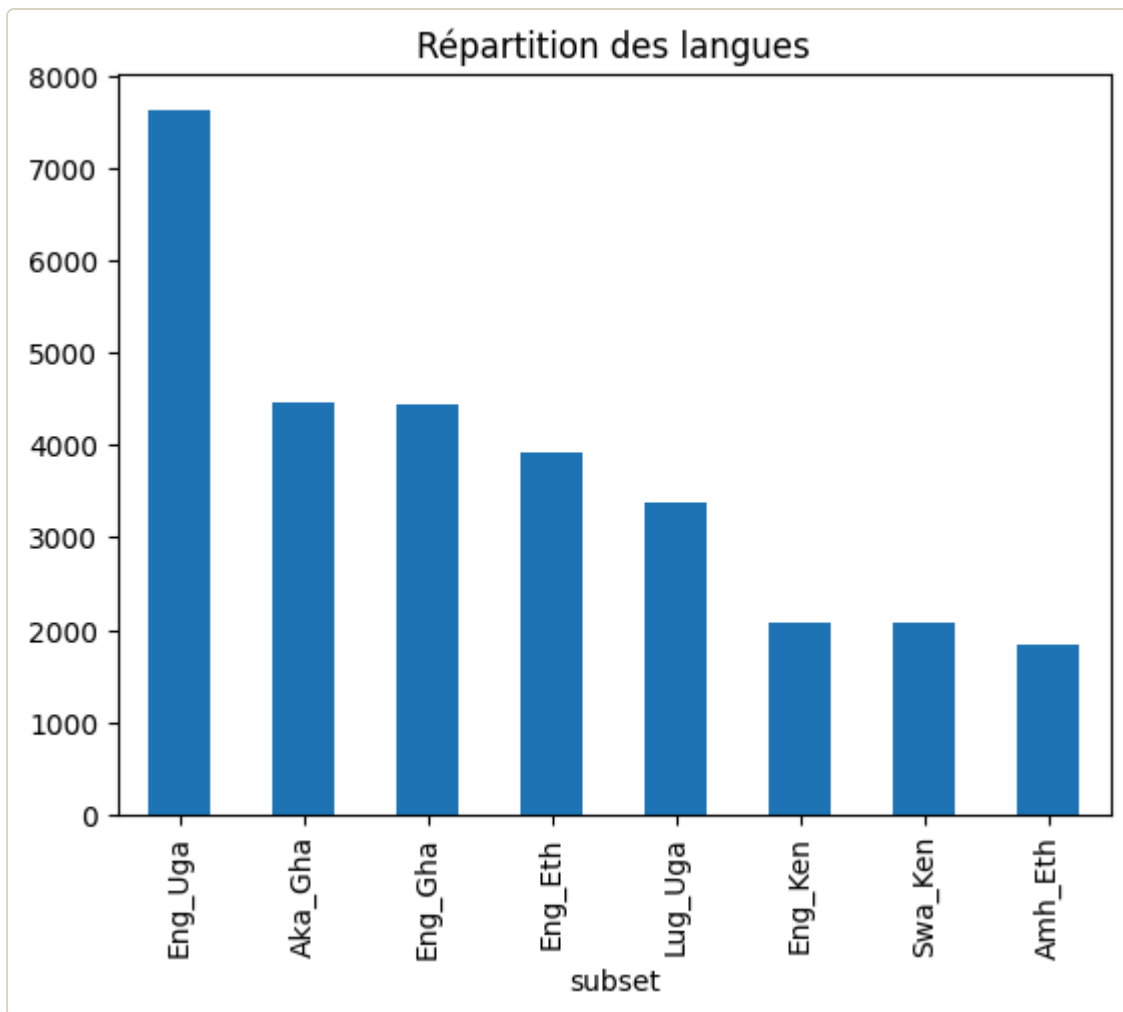
Constat — Les trois jeux sont propres structurellement : aucune valeur manquante, aucun ID dupliqué, types cohérents. Nous n'avons donc eu aucun nettoyage structurel à faire avant la Phase 2 — notre travail de préparation a pu se concentrer sur le contenu (déséquilibre entre langues, longueurs des textes).

3.2 Distribution des langues

Le dataset présente un déséquilibre net entre les 8 sous-ensembles langue/pays : la langue la plus représentée (Eng_Uga) pèse plus de 4 fois la moins représentée (Amh_Eth).



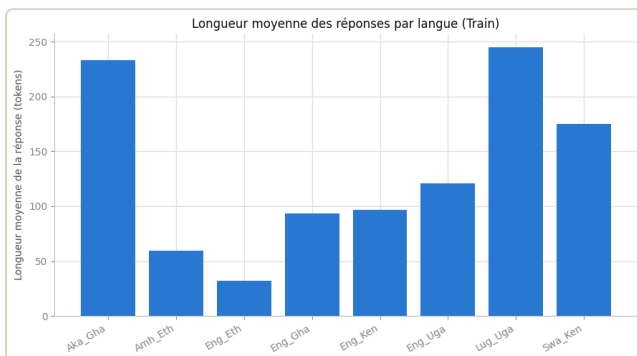
Distribution des langues, Train vs Val (pipeline Gemma).



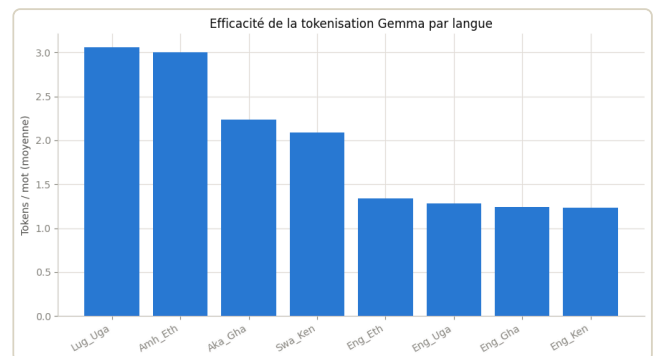
Même constat, retrouvé indépendamment dans le pipeline mT5.

C'est ce déséquilibre qui nous a poussés, dans le pipeline Gemma, à sur-échantillonner les langues minoritaires avant l'entraînement (section 5.2).

3.3 Longueur des séquences et efficacité de la tokenisation



Longueur moyenne des réponses (tokens) par langue.



Efficacité de la tokenisation Gemma (tokens/mot).



Constat — Le tokenizer fragmente fortement les langues africaines peu dotées en ressources (luganda 3,06 tokens/mot, amharique 3,01, akan 2,24, swahili 2,09), contre un ratio proche de 1,2–1,3 pour les variantes en anglais. Nous en avons déduit que ces langues sont peu représentées dans le vocabulaire des tokeniseurs pré-entraînés — un signal que nous avons retrouvé plus tard, de façon cohérente, dans les résultats d'évaluation des deux modèles.

4. Modèle 1 — mT5-small (full fine-tuning)

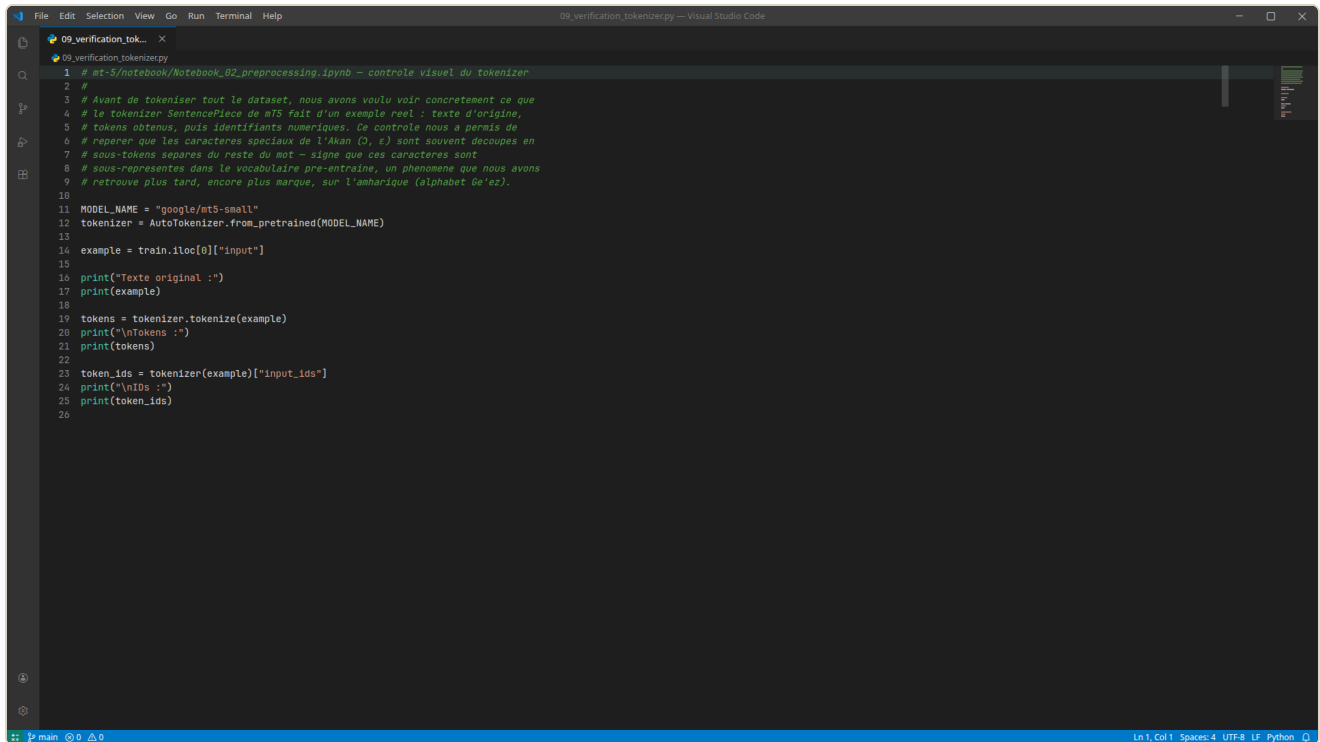
Le premier modèle que nous avons entraîné est **mT5-small** (300 millions de paramètres), en *full fine-tuning* — un choix cohérent avec sa taille raisonnable, qui rend cette approche abordable sur un seul GPU Colab, contrairement à Gemma E2B (section 5).

4.1 Pré-traitement

Nous avons structuré la préparation du jeu de données mT5 en un pipeline en six étapes, du CSV brut jusqu'aux identifiants numériques exploitables par le modèle :

- 1 Charger les données
CSV → Pandas
- 2 Explorer et contrôler la qualité
valeurs manquantes, doublons, dimensions
- 3 Nettoyer
espaces superflus + normalisation Unicode NFC
- 4 Convertir
DataFrame → Hugging Face Dataset
- 5 Charger le tokenizer mT5
google/mt5-small, SentencePiece, 101 langues
- 6 Tokeniser
questions → `input_ids`, réponses → `labels`

Nous avons normalisé les caractères en Unicode NFC avant tokenisation, pour éviter que deux textes visuellement identiques soient tokenisés différemment selon leur encodage d'origine — un risque réel vu le mélange de systèmes d'écriture de notre dataset (latin, Ge'ez).



```
09_verification_tok... X
09_verification_tokenizer.py
1 # mt-5/notebook/Notebook_02_preprocessing.ipynb - controle visuel du tokenizer
2 #
3 # Avant de tokeniser tout le dataset, nous avons voulu voir concretement ce que
4 # le tokenizer SentencePiece de mT5 fait d'un exemple reel : texte d'origine,
5 # tokens obtenus, puis identifiants numeriques. Ce controle nous a permis de
6 # repérer que les caracteres speciaux de l'Akan (ɔ, ε) sont souvent decoupes en
7 # sous-tokens separees du reste du mot - signe que ces caracteres sont
8 # sous-representes dans le vocabulaire pre-entraîne, un phenomene que nous avons
9 # retrouve plus tard, encore plus marque, sur l'amharique (alphabet Ge'ez).
10
11 MODEL_NAME = "google/mt5-small"
12 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
13
14 example = train.iloc[0]["input"]
15
16 print("Texte original :")
17 print(example)
18
19 tokens = tokenizer.tokenize(example)
20 print("\ntokens :")
21 print(tokens)
22
23 token_ids = tokenizer(example)["input_ids"]
24 print("\ntIDs :")
25 print(token_ids)
26
```

Extrait — contrôle visuel du tokenizer sur un exemple réel avant de l'appliquer à tout le dataset.

Avant de tokeniser l'ensemble du dataset, nous avons voulu voir concrètement ce que le tokenizer SentencePiece de mT5 fait d'un exemple réel : texte d'origine, tokens obtenus, puis identifiants numériques. Ce contrôle nous a permis de repérer que les caractères spéciaux de l'akan (ɔ, ε) sont souvent découpés en sous-tokens séparés du reste du mot — signe que ces caractères sont sous-représentés dans le vocabulaire pré-entraîné, un phénomène que nous avons retrouvé plus tard, encore plus marqué, sur l'amharique (alphabet Ge'ez) et que nous relierons directement au constat de la section 3.3.

Nous avons fixé une longueur maximale de 128 tokens pour les questions et 256 tokens pour les réponses, avec troncature au-delà — un choix directement dérivé des longueurs mesurées en section 3.3 : les réponses sont systématiquement plus longues que les questions (voir la figure « Longueur moyenne des réponses par langue »), il fallait donc leur laisser nettement plus de marge qu'aux questions. Le plafond de 256 couvre la grande majorité des réponses observées sans pour autant gonfler inutilement les séquences les plus courtes.

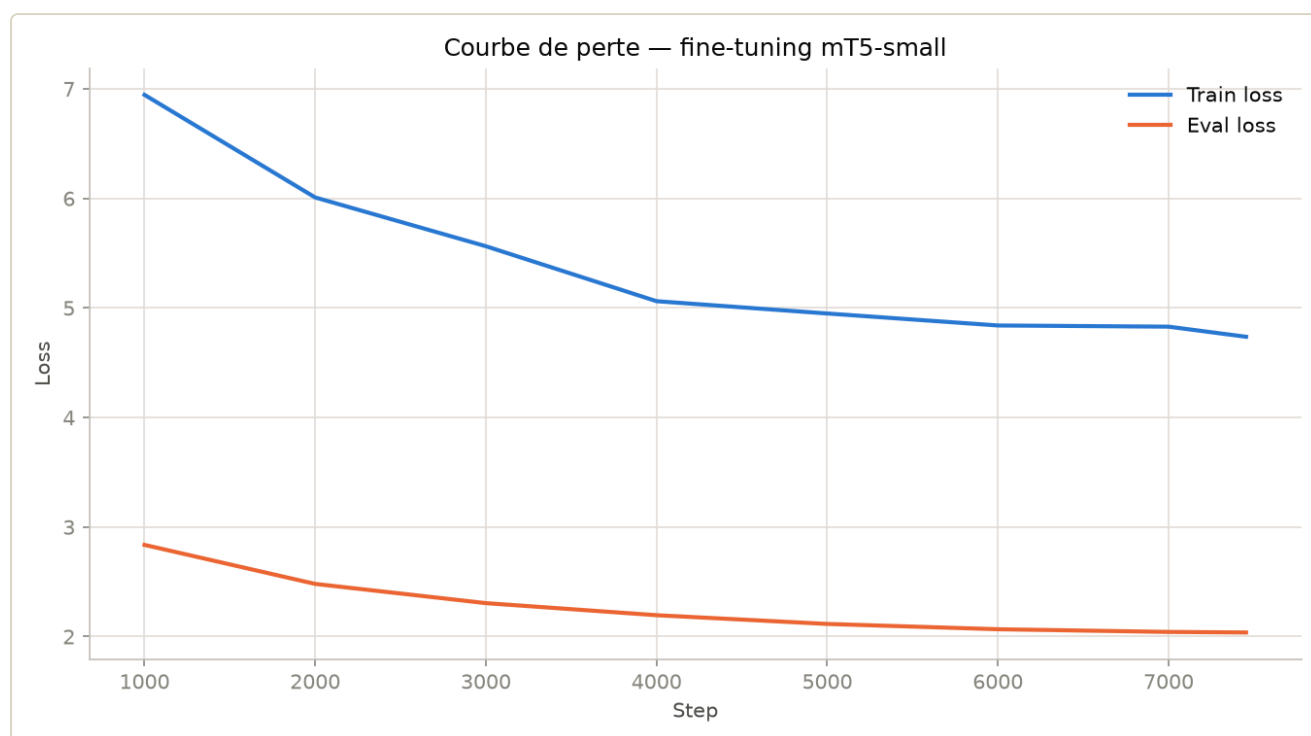
4.2 Entraînement

Paramètre	Valeur
Epochs	2
Batch size (train/eval)	4 par device
Gradient accumulation	2 (batch effectif = 8)
Learning rate	3×10^{-4} (avec warmup)
Weight decay	0,01
Optimiseur	Adafactor
Précision	FP16/BF16 désactivés

Nous avons choisi l'optimiseur **Adafactor** plutôt qu'AdamW parce que c'est l'optimiseur historiquement utilisé pour pré-entraîner toute la famille T5/mT5 — rester cohérent avec l'optimiseur d'origine nous a semblé plus sûr qu'un choix générique. C'est aussi ce qui a guidé notre choix de `learning_rate=3×10-4` : c'est la valeur reprise par les scripts officiels Hugging Face de fine-tuning T5/mT5 (`run_translation.py`, `run_summarization.py`) spécifiquement pour un entraînement avec Adafactor — une valeur déjà éprouvée sur ce couple modèle/optimiseur, que nous avons préférée à un balayage de taux d'apprentissage que notre budget de calcul (un seul GPU Colab) ne permettait pas de mener sérieusement. Nous avons désactivé le calcul en précision réduite (FP16/BF16) après avoir constaté que mT5 y est numériquement instable ; le T4 fourni par Colab ne gère de toute façon pas le bf16 efficacement. Le batch effectif de 8 (4 par device × 2 d'accumulation) est un compromis pour limiter la consommation mémoire du T4 tout en gardant un batch statistiquement raisonnable.

Évolution de la loss pendant l'entraînement (7 454 steps, 2 epochs).

Step	Training Loss	Validation Loss
1 000	6,9456	2,8372
2 000	6,0090	2,4809
3 000	5,5638	2,3054
4 000	5,0619	2,1952
5 000	4,9497	2,1155
6 000	4,8403	2,0671
7 000	4,8294	2,0425
7 454	4,7372	2,0376



Courbe de perte — 7 454 steps, 2 epochs, ~2h33 d'entraînement sur GPU T4.



Constat — La training loss diminue de 6,95 à 4,74 et la validation loss de 2,84 à 2,04. Sur toute la période observée, **aucune remontée de la validation loss** ne se manifeste — contrairement à ce que nous avons observé sur Gemma (section 5.3), le modèle continue d'améliorer la fonction objectif jusqu'à la fin des 2 epochs, sans signe de sur-apprentissage sur cette plage de mesure.

4.3 Test qualitatif immédiat

Question	Réponse générée
Can I breastfeed if I have a fever?	This is a question about, General. No. Breastfeeding is a healthy and healthy diet.

Ce test nous a confirmé que le pipeline fonctionne techniquement de bout en bout, mais nous a aussi montré que la qualité restait limitée sur cet exemple isolé — la formulation est incohérente et la réponse « No » n'est pas justifiée. Nous avons donc jugé nécessaire de poursuivre avec une évaluation systématique sur l'ensemble du jeu de validation.

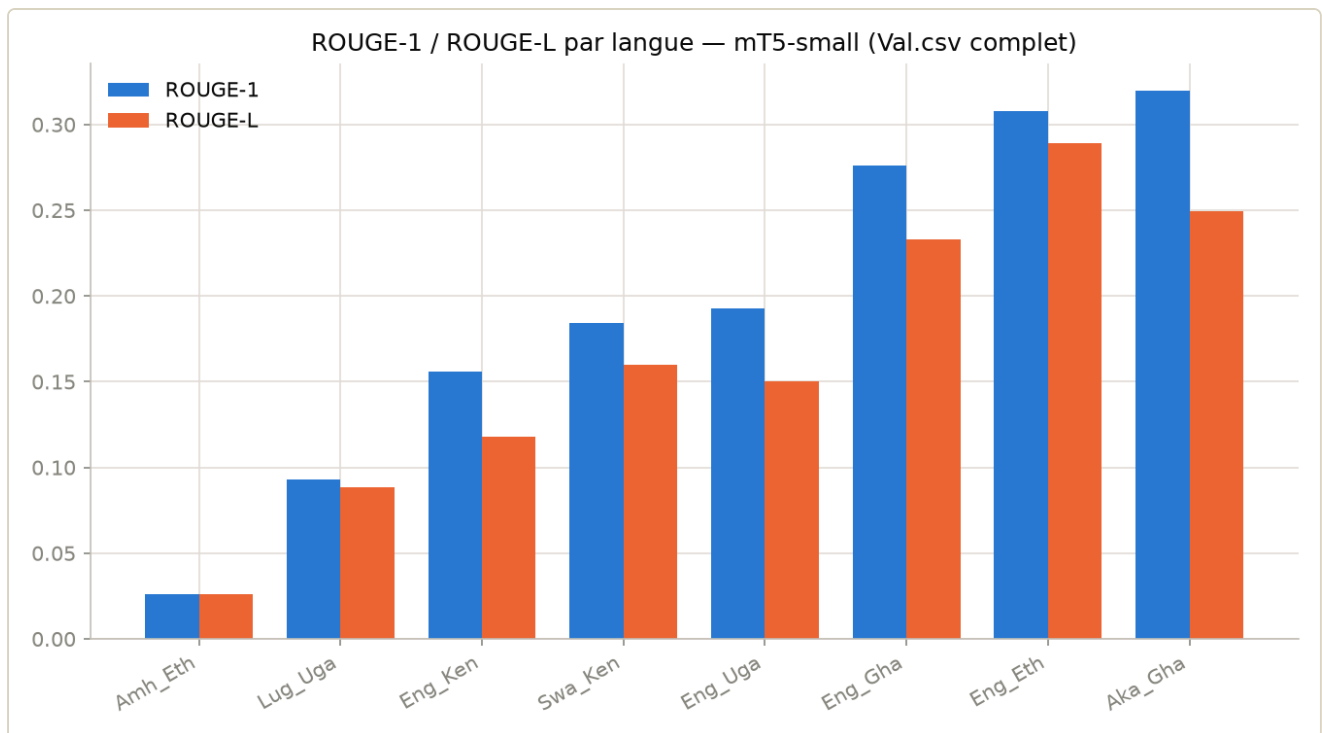
4.4 Évaluation sur le jeu de validation

Nous avons généré une réponse pour chacune des 6 686 questions de validation, par lots de 8, avec une recherche en faisceau (`num_beams=4`) plutôt qu'un décodage glouton, pour privilégier la qualité des réponses générées.

Métrique	Score F1 moyen
ROUGE-1	0,2104
ROUGE-L	0,1745

Résultats par langue.

Langue	ROUGE-1	ROUGE-L
Amh_Eth	0,0259	0,0259
Lug_Uga	0,0929	0,0884
Eng_Ken	0,1560	0,1182
Swa_Ken	0,1843	0,1596
Eng_Uga	0,1926	0,1501
Eng_Gha	0,2759	0,2329
Eng_Eth	0,3078	0,2889
Aka_Gha	0,3199	0,2492



ROUGE-1 et ROUGE-L par langue — mT5-small (jeu de validation complet).



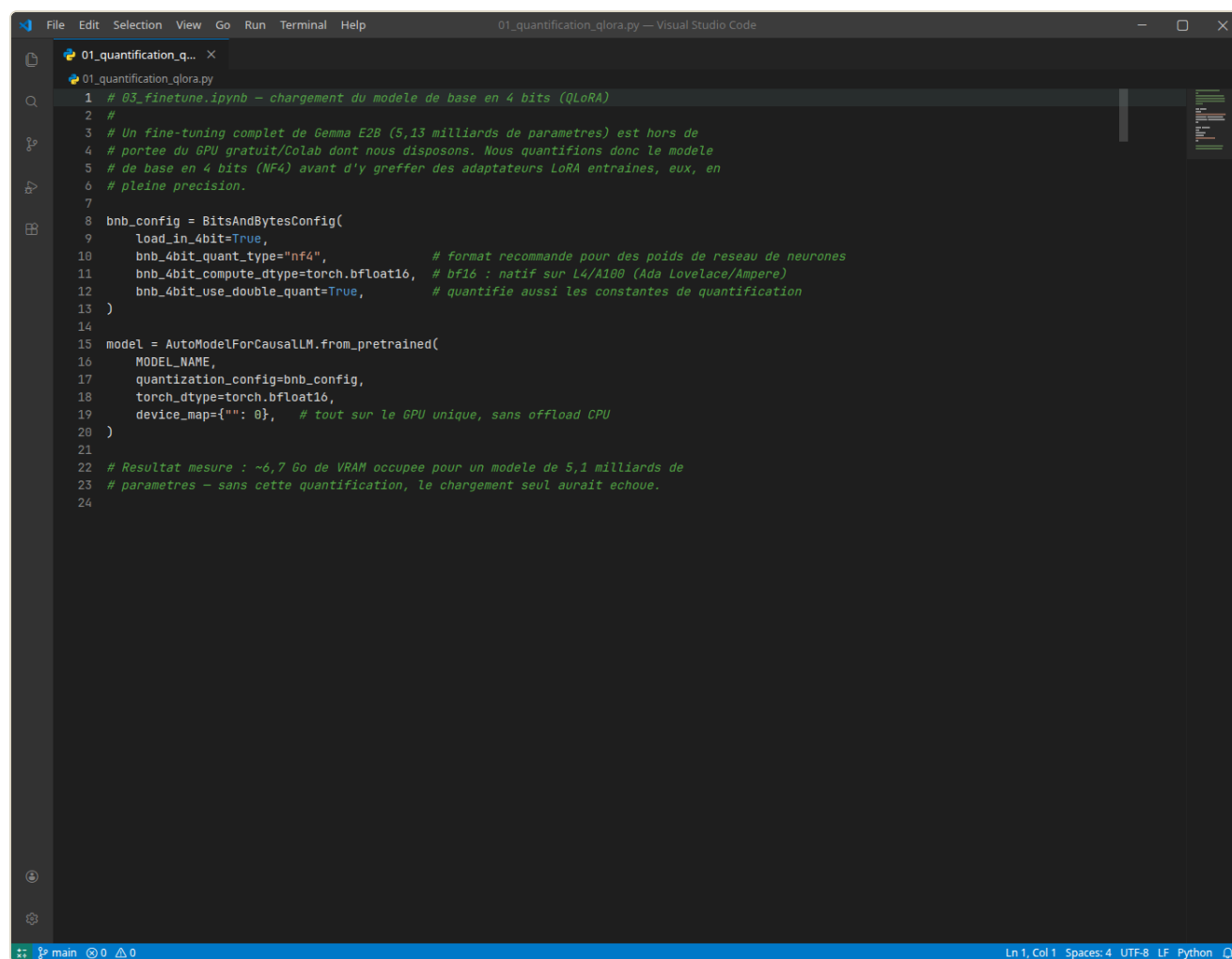
Constat — L'écart entre la meilleure langue (Aka_Gha, 0,32) et la pire (Amh_Eth, 0,026) est d'un facteur ~ 12 . Nous expliquons principalement cette disparité par le script d'écriture de l'amharique (Ge'ez), très éloigné des scripts latins des autres langues et probablement sous-représenté dans le pré-entraînement de mT5 — une explication cohérente avec le ratio tokens/mot le plus élevé déjà relevé en section 3.3. En inspectant les réponses générées en amharique, nous avons identifié trois symptômes récurrents : répétition de la question au lieu d'y répondre, bascule de langue vers l'anglais en cours de génération, et réponses tautologiques ou en boucle.

Ce que nous en retenons pour une itération future : le modèle fonctionne de façon satisfaisante sur les langues à script latin (akan, anglais, dans une moindre mesure swahili), avec un ROUGE-1 dans la fourchette 0,15–0,32. L'amharique reste le point faible critique et tire la moyenne globale vers le bas — une amélioration ciblée sur cette langue (sur-échantillonnage, epochs supplémentaires, ou vérification du support du script Ge'ez par le tokeniseur) aurait selon nous le plus fort impact sur le score final.

5. Modèle 2 — Gemma E2B (QLoRA)

Le second modèle que nous avons testé est **Gemma E2B** (google/gemma-4-E2B-it), environ 5,13 milliards de paramètres — bien plus volumineux que mT5. Compte tenu de cette taille, un fine-tuning complet était hors de portée des GPU dont nous disposions (T4 puis L4 sur Colab), aussi bien en mémoire qu'en temps de calcul : nous avons donc choisi **QLoRA**, détaillée section par section ci-dessous.

5.1 Chargement en 4 bits



```
01_quantification_qlora.py
1 # 03_finetune.ipynb - chargement du modele de base en 4 bits (QLoRA)
2 #
3 # Un fine-tuning complet de Gemma E2B (5,13 milliards de parametres) est hors de
4 # portee du GPU gratuit/Colab dont nous disposons. Nous quantifions donc le modele
5 # de base en 4 bits (NF4) avant d'y greffer des adaptateurs LoRA entraines, eux, en
6 # pleine precision.
7
8 bnb_config = BitsAndBytesConfig(
9     load_in_4bit=True,
10     bnb_4bit_quant_type="nf4",           # format recommande pour des poids de reseau de neurones
11     bnb_4bit_compute_dtype=torch.bfloat16, # bfloat16 : natif sur L4/A100 (Ada Lovelace/Ampere)
12     bnb_4bit_use_double_quant=True,      # quantifie aussi les constantes de quantification
13 )
14
15 model = AutoModelForCausalLM.from_pretrained(
16     MODEL_NAME,
17     quantization_config=bnb_config,
18     torch_dtype=torch.bfloat16,
19     device_map={"": 0}, # tout sur le GPU unique, sans offload CPU
20 )
21
22 # Resultat mesure : ~6,7 Go de VRAM occupee pour un modele de 5,1 milliards de
23 # parametres - sans cette quantification, le chargement seul aurait echoue.
24
```

Extrait — chargement du modèle de base en 4 bits (NF4).

Nous avons chargé le modèle sur un GPU **L4** (24 Go de VRAM), après avoir commencé le setup initial sur un T4. Ce changement était motivé par deux choses : le L4 supporte nativement le bfloat16 (architecture Ada Lovelace), contrairement au T4 (Turing), et il offre davantage de marge

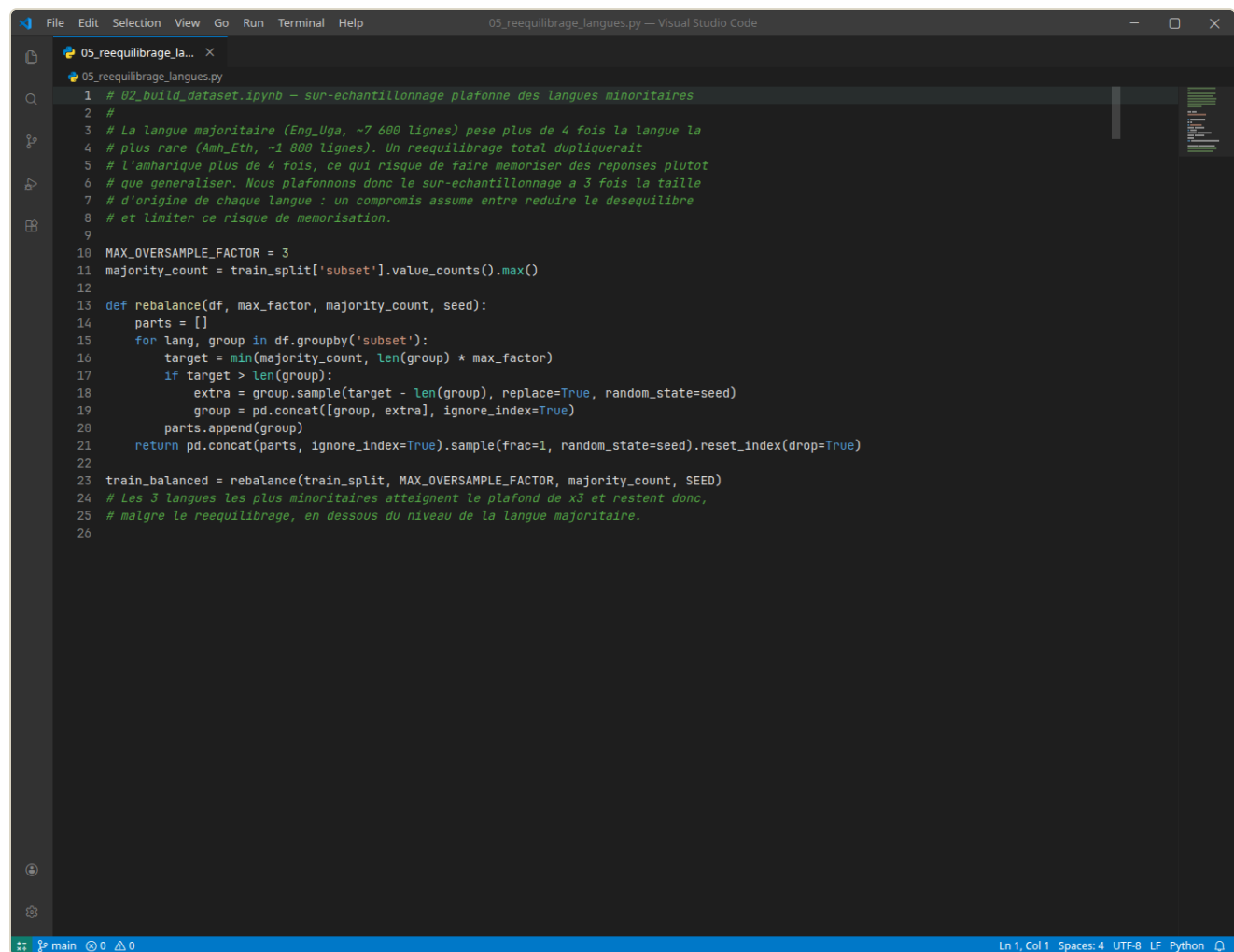
mémoire. Résultat mesuré : ~6,7 Go de VRAM occupée pour charger un modèle de plus de 5 milliards de paramètres — sans la quantification 4-bit, ce chargement aurait simplement échoué sur le matériel dont nous disposions.

5.2 Construction du dataset d'instruction-tuning

Nous avons mis chaque exemple au format « chat » attendu par Gemma. Le prompt d'instruction est rédigé **en français**, mais demande explicitement une réponse dans la langue locale de la question — le modèle est donc entraîné à comprendre une instruction en français et à répondre dans la langue africaine ciblée, jamais en français.

Nous avons ensuite réservé 5 % du jeu d'entraînement (stratifié par langue) comme jeu de développement interne, pour surveiller le sur-apprentissage pendant l'entraînement sans jamais toucher à `Val.csv`, que nous voulions garder intact comme validation finale fidèle à l'évaluation Zindi.

Rééquilibrage des langues minoritaires



```
05_reequilibrage_langues.py — Visual Studio Code
05_reequilibrage_langues.py
1 # 02_build_dataset.ipynb — sur-échantillonnage plafonne des langues minoritaires
2 #
3 # La langue majoritaire (Eng_Uga, ~7 600 lignes) pese plus de 4 fois la langue la
4 # plus rare (Amh_Eth, ~1 800 lignes). Un reequilibrage total dupliquerait
5 # l'amharique plus de 4 fois, ce qui risque de faire memoriser des reponses plutot
6 # que generaliser. Nous plafonnons donc le sur-échantillonnage a 3 fois la taille
7 # d'origine de chaque langue : un compromis assume entre reduire le desequilibre
8 # et limiter ce risque de memorisation.
9
10 MAX_OVERSAMPLE_FACTOR = 3
11 majority_count = train_split['subset'].value_counts().max()
12
13 def rebalance(df, max_factor, majority_count, seed):
14     parts = []
15     for lang, group in df.groupby('subset'):
16         target = min(majority_count, len(group) * max_factor)
17         if target > len(group):
18             extra = group.sample(target - len(group), replace=True, random_state=seed)
19             group = pd.concat([group, extra], ignore_index=True)
20         parts.append(group)
21     return pd.concat(parts, ignore_index=True).sample(frac=1, random_state=seed).reset_index(drop=True)
22
23 train_balanced = rebalance(train_split, MAX_OVERSAMPLE_FACTOR, majority_count, SEED)
24 # Les 3 langues les plus minoritaires atteignent le plafond de x3 et restent donc,
25 # malgre le reequilibrage, en dessous du niveau de la langue majoritaire.
26
```

Extrait — sur-échantillonnage plafonné à $\times 3$.

Langue	Avant	Après	Facteur
Eng_Uga	7 243	7 243	×1 (référence)
Aka_Gha	4 232	7 243	×1,71
Eng_Gha	4 221	7 243	×1,72
Eng_Eth	3 719	7 243	×1,95
Lug_Uga	3 214	7 243	×2,25
Eng_Ken	1 976	5 928	×3 (plafond)
Swa_Ken	1 966	5 898	×3 (plafond)
Amh_Eth	1 753	5 259	×3 (plafond)

Nous avons délibérément plafonné le sur-échantillonnage à 3 fois la taille d'origine de chaque langue, plutôt que de viser un rééquilibrage parfait. La raison : ramener l'amharique (1 753 lignes) au niveau de la langue majoritaire (7 243) l'aurait dupliqué plus de 4 fois, avec un risque réel que le modèle mémorise ces réponses au lieu d'apprendre à généraliser — un risque que nous avons jugé plus grave que le déséquilibre résiduel accepté par ce plafond.

5.3 Fine-tuning LoRA/QLoRA

```

1 # Hf, finetune.py - configuration des adaptateurs LoRA
2 #
3 # Nous gelons entièrement le modèle de base et n'entraînez que de petites matrices
4 # ajoutées sur les couches d'attention et du MLP. Nous avons retenu r=16 : une valeur
5 # stable dans la littérature LoRA, qui offre un compromis raisonnable entre précision
6 # d'adaptation et nombre de paramètres entraînés.
7
8 for param in model.parameters():
9     param.requires_grad = False
10
11 lora_config = LoraConfig(
12     r=16,
13     lora_alpha=32,
14     lora_dropout=0.1,
15     bias="none",
16     task_type="LORA",
17     target_modules=[
18         "q_proj", "k_proj", "v_proj", "o_proj",
19         "gate_proj", "up_proj", "down_proj",
20     ],
21 )
22 model = get_peft_model(model, lora_config)
23 model.print_trainable_parameters()
24
25 # Résultats obtenus
26 # Paramètres entraînés : 29.859 Mo
27 # Paramètres totaux : 0.134 127 Mo
28 # Proportion entraînée : 0.50 %
29

```

Extrait — configuration des adaptateurs LoRA.

```

1 # Hf, finetune.py - contournement d'une particularité du checkpoint Gemma F16
2 #
3 # Les projections d'attention de ce checkpoint sont développées dans une classe
4 # GemmaAttentionLinear (classe des activations) que PEFT ne reconnaît pas pour r
5 # entraîneur le modèle LoRA. Nous la remplaçons vers une LinearAttn avant
6 # de configurer LoRA - ce qui désactive le clipping pendant l'entraînement.
7 # A surveiller : si le loss divergeait ou produisait des NaN, ce clipping serait
8 # probablement nécessaire et annulerait une solution plus fine.
9
10 def unwrap_clippable_linear(model):
11     count = 0
12     for module in model.modules():
13         for child_name, child in list(module.named_children()):
14             if child.__class__.__name__ == "GemmaClippableLinear":
15                 setattr(module, child_name, child.linear)
16                 count += 1
17     print(f"Count {count} couches GemmaClippableLinear déballées vers LinearAttn")
18     return model
19
20 model = unwrap_clippable_linear(model)
21 # 22 couches attention - le loss d'entraînement a divergé normalement.
22 # Le risque signalé ne s'est pas matérialisé sur ce run.
23

```

Extrait — contournement d'une particularité du checkpoint.

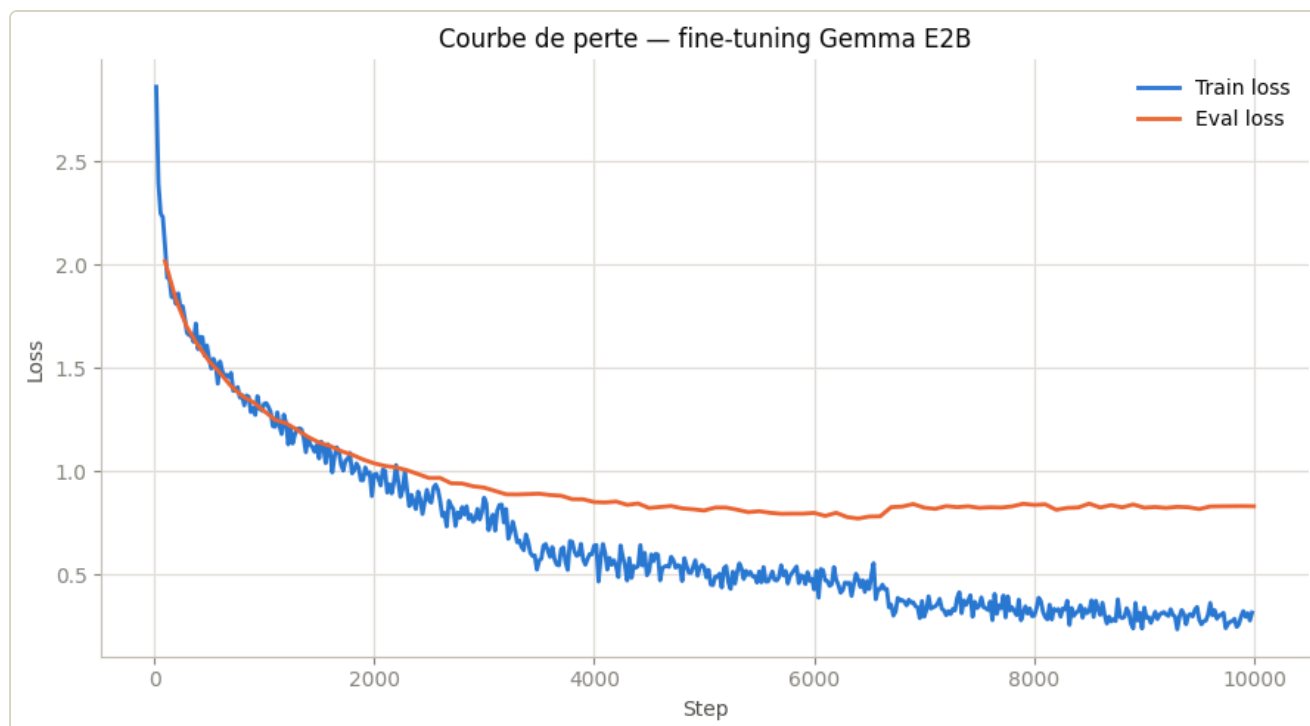
Nous avons gelé entièrement le modèle de base et n'avons entraîné que les adaptateurs LoRA, avec un rang $r=16$. Ce rang nous semblait le bon compromis pour notre cas : assez élevé pour donner à l'adaptateur une vraie capacité d'apprentissage sur une tâche *générative* multilingue (contrairement à des tâches de classification plus étroites, souvent traitées avec $r=4$ ou $r=8$ dans la littérature), mais sans multiplier inutilement les 29,9 millions de paramètres entraînables sur un budget de calcul limité à un seul GPU Colab, où chaque run supplémentaire pour tester un rang différent représentait plusieurs heures. Nous avons fixé `lora_alpha=32`, soit exactement le double du rang ($\alpha/r=2$) : ce ratio fixe le facteur d'échelle de la mise à jour LoRA ($\Delta W = (\alpha/r) \cdot BA$) à une valeur par défaut largement validée dans les exemples officiels PEFT, ce qui nous a évité d'avoir à faire un balayage séparé de ce paramètre en plus du rang.

Paramètres	Valeur
Paramètres entraînables	29 859 840
Paramètres totaux	5 134 157 344
Proportion entraînable	0,58 %

```
File Edit Selection View Go Run Terminal Help 04_prompt_masking_collator.py — Visual Studio Code
04_prompt_masking_collator.py
1 # 03_finetune.ipynb — masquage de la question dans le calcul de la perte
2 #
3 # full_text = prompt + reponse ; prompt_text = prompt seul. Nous tokenisons les
4 # deux et masquons (label = -100) les tokens du prompt dans full_text, pour que la
5 # perte ne porte que sur la reponse generée — sans dependre du texte exact des
6 # tokens speciaux du template (qui s'est avère être <|turn>...<|turn> pour ce
7 # checkpoint, différent du <start_of_turn> d'autres versions de Gemma).
8
9 class PromptMaskingCollator:
10     def __init__(self, tokenizer, max_length):
11         self.tokenizer = tokenizer
12         self.max_length = max_length
13
14     def __call__(self, batch):
15         full_texts = [ex['full_text'] for ex in batch]
16         prompt_texts = [ex['prompt_text'] for ex in batch]
17
18         enc = self.tokenizer(
19             full_texts, add_special_tokens=False, truncation=True,
20             max_length=self.max_length, padding=True, return_tensors='pt',
21         )
22         labels = enc['input_ids'].clone()
23
24         for i, prompt in enumerate(prompt_texts):
25             prompt_len = len(self.tokenizer(prompt, add_special_tokens=False)['input_ids'])
26             prompt_len = min(prompt_len, labels.shape[1])
27             labels[i, :prompt_len] = -100
28         labels[enc['attention_mask'] == 0] = -100
29
30         return {'input_ids': enc['input_ids'], 'attention_mask': enc['attention_mask'], 'labels': labels}
31
```

Extrait — masquage de la question dans le calcul de la perte.

Nous avons construit un collator qui masque les tokens de la question dans le calcul de la perte, pour que le modèle ne soit noté et corrigé que sur sa capacité à produire la réponse — jamais sur sa capacité à reproduire la question. Nous avons choisi de mesurer la longueur du prompt plutôt que de chercher les tokens spéciaux du template, car ce checkpoint utilise `<|turn>...<|turn>`, différent du `<start_of_turn>` d'autres versions de Gemma — une mesure par longueur nous semblait plus robuste face à cette variation.



Courbe de perte — 9 996 steps, 3 epochs, 16h45 d'entraînement sur GPU L4.



Constat — Nous observons un **sur-apprentissage net à partir d'environ le step 6 400**. La loss de validation atteint son minimum ($\approx 0,772$) à ce point, puis remonte et se stabilise en plateau ($0,78\text{--}0,84$) jusqu'à la fin, tandis que la loss d'entraînement continue de chuter jusqu'à $\sim 0,24\text{--}0,32$. Grâce à `load_best_model_at_end=True`, le checkpoint réellement conservé correspond au step $\sim 6\,400$, pas à la fin de l'entraînement — les $\sim 5\text{--}6$ dernières heures de calcul ont donc surtout produit du sur-apprentissage sans bénéfice, un point que nous retenons pour un futur run (arrêt anticipé via `early_stopping`).

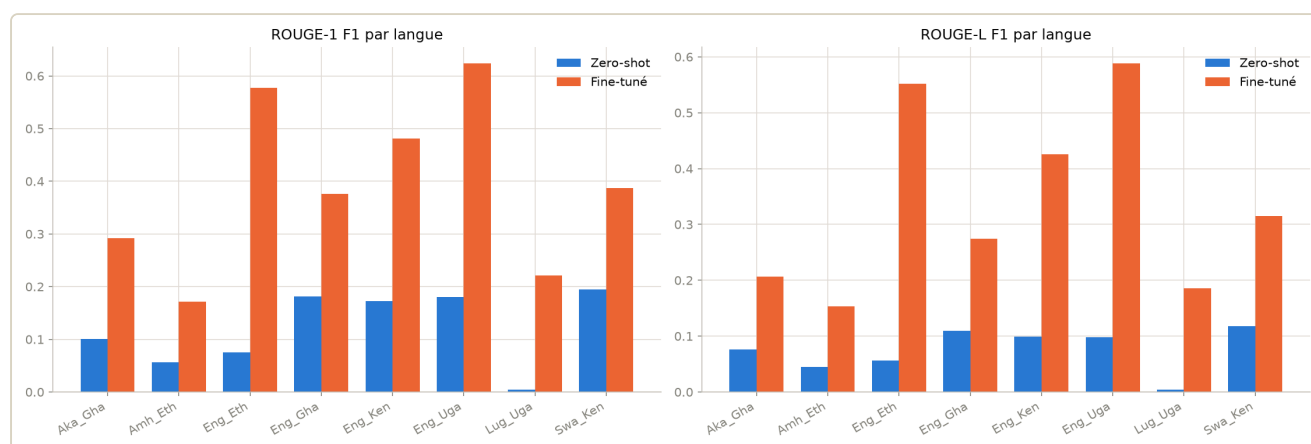
5.4 Évaluation (zero-shot vs fine-tuné)

Comme pour mT5 (section 4.4), nous avons généré une réponse pour chacune des 6 686 questions de `Val.csv`, avec et sans l'adaptateur LoRA, ce qui rend la comparaison entre les deux modèles pleinement rigoureuse — les deux portent désormais sur le même jeu complet.

Métrique	Zero-shot	Fine-tuné	Gain
ROUGE-1 (F1)	0,1280	0,4149	+0,287 ($\times 3,2$)
ROUGE-L (F1)	0,0791	0,3583	+0,279 ($\times 4,5$)

Résultats par langue (jeu de validation complet).

Langue	ROUGE-1 zero-shot	ROUGE-1 fine-tuné	ROUGE-L zero-shot	ROUGE-L fine-tuné
Amh_Eth	0,0566	0,1713	0,0452	0,1537
Lug_Uga	0,0050	0,2215	0,0046	0,1859
Aka_Gha	0,1006	0,2917	0,0765	0,2067
Eng_Gha	0,1811	0,3758	0,1094	0,2748
Swa_Ken	0,1949	0,3873	0,1180	0,3147
Eng_Ken	0,1722	0,4813	0,0990	0,4252
Eng_Eth	0,0748	0,5775	0,0564	0,5519
Eng_Uga	0,1798	0,6241	0,0985	0,5886



ROUGE-1 et ROUGE-L par langue — zero-shot vs fine-tuné (jeu de validation complet, 6 686 exemples).



Constat — Le fine-tuning apporte un gain net et sans ambiguïté sur l'ensemble du jeu de validation, avec un ROUGE-L multiplié par 4,5 — un gain même supérieur à celui que laissait présager notre premier run sur un échantillon de 500 exemples (ROUGE-1 fine-tuné 0,37 sur l'échantillon contre 0,41 sur le jeu complet) : l'échantillon initial sous-estimait légèrement la qualité réelle du modèle fine-tuné. Le classement par langue confirme un constat déjà établi avec les trois autres approches (sections 2.1, 2.4, 4.4) : **l'amharique reste la langue la plus faible** (ROUGE-1 fine-tuné 0,17), tandis qu'**Eng_Uga**, la langue la mieux dotée du jeu de données, obtient le meilleur score (0,62).

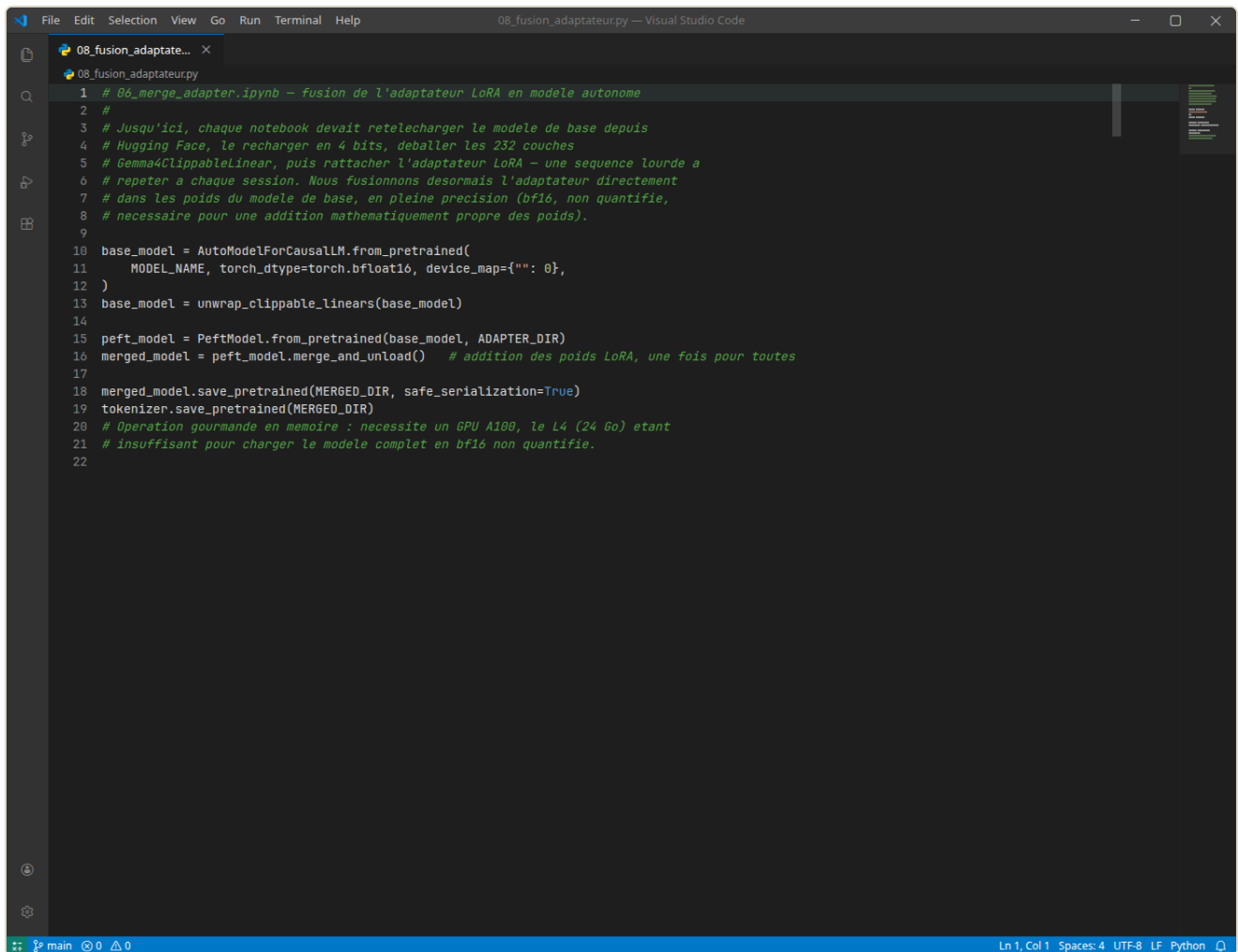


Constat — Le cas le plus frappant est le luganda en zero-shot : ROUGE-1 = 0,005, quasiment nul, bien plus bas que les autres langues en zero-shot (0,06 à 0,19). En inspectant les prédictions, nous avons compris pourquoi : plutôt que de produire une réponse hors sujet ou dans la mauvaise langue comme pour les autres langues, le modèle de base **boucle sur un mot répété** (par exemple « n'okwela n'okwela n'okwela... » ou « kye kye kye... »), un mode de dégénérescence classique d'un modèle générique confronté à une langue quasi absente de son pré-entraînement. Le fine-tuning corrige presque entièrement ce problème (ROUGE-1 0,005 → 0,22), ce qui montre que le modèle peut apprendre à produire du luganda correct malgré une base de départ qui n'en connaît pratiquement rien. Nous avons par ailleurs observé, comme sur l'échantillon initial, que le zero-shot souffre aussi de changements de langue non désirés et de méta-commentaires parasites (« Voici une réponse en Akan... »), et que le fine-tuné reproduit occasionnellement un artefact du modèle de base sur un exemple en anglais (référence à « ma dernière mise à jour de connaissances en septembre 2021 »), signe que certains réflexes du pré-entraînement générique subsistent malgré le fine-tuning.

5.5 Génération de la soumission finale

Nous avons généré une réponse pour l'intégralité des 2 618 questions de `Test.csv`, par lots de 8, en décodage déterministe. Nous avons généré et validé le fichier de soumission : 2 618 lignes, sans ID manquant ni dupliqué, sans réponse vide. Conformément au format attendu par Zindi, la même réponse générée est dupliquée dans les trois colonnes cibles (`TargetRLF1`, `TargetR1F1`, `TargetLLM`), puisque notre modèle produit une seule réponse par question, évaluée ensuite selon trois métriques différentes côté plateforme.

5.6 Fusion de l'adaptateur en modèle autonome



```
08_fusion_adaptateur.py
1 # 06_merge_adapter.ipynb — fusion de l'adaptateur LoRA en modèle autonome
2 #
3 # Jusqu'ici, chaque notebook devait retelecharger le modèle de base depuis
4 # Hugging Face, le recharger en 4 bits, déballer les 232 couches
5 # Gemma4ClippableLinear, puis rattacher l'adaptateur LoRA — une séquence lourde à
6 # répéter à chaque session. Nous fusionnons désormais l'adaptateur directement
7 # dans les poids du modèle de base, en pleine précision (bf16, non quantifié,
8 # nécessaire pour une addition mathématiquement propre des poids).
9
10 base_model = AutoModelForCausalLM.from_pretrained(
11     MODEL_NAME, torch_dtype=torch.bfloat16, device_map={"": 0},
12 )
13 base_model = unwrap_clippable_linears(base_model)
14
15 peft_model = PeftModel.from_pretrained(base_model, ADAPTER_DIR)
16 merged_model = peft_model.merge_and_unload() # addition des poids LoRA, une fois pour toutes
17
18 merged_model.save_pretrained(MERGED_DIR, safe_serialization=True)
19 tokenizer.save_pretrained(MERGED_DIR)
20 # Operation gourmande en memoire : necessite un GPU A100, le L4 (24 Go) etant
21 # insuffisant pour charger le modele complet en bf16 non quantifie.
22
```

Extrait — fusion de l'adaptateur dans les poids du modèle de base.

Nous avons ajouté une dernière étape, de nature infrastructurelle plutôt qu'analytique. Jusque-là, chaque notebook devait retélécharger le modèle de base depuis Hugging Face, le recharger en 4 bits, puis déballer manuellement les 232 couches `Gemma4ClippableLinear` avant de rattacher l'adaptateur — une séquence lourde à répéter à chaque session. Nous avons donc fusionné l'adaptateur LoRA directement dans les poids du modèle de base, en pleine précision (bf16, non quantifié, nécessaire pour une addition mathématiquement propre des poids). Cette opération, gourmande en mémoire (10,22 Go alloués pour la fusion à elle seule, en plus du modèle bf16 déjà chargé), nous a demandé un GPU **A100 80 Go**, le L4 (24 Go) étant insuffisant pour charger le modèle complet en bf16. Le modèle fusionné (10,24 Go, cohérent avec les ~5,13 milliards de paramètres en bf16) est sauvegardé sur Drive et n'a plus besoin ni de Hugging Face ni de l'adaptateur séparé pour être rechargé.

Nous avons vérifié la fusion en rechargeant ce modèle directement depuis Drive (quantifié à nouveau en 4 bits, sans `PeftModel` ni connexion Hugging Face) puis en générant une réponse test : le modèle répond correctement et dans la bonne langue sur un exemple de santé en swahili, confirmant que la fusion préserve bien le comportement du modèle fine-tuné. Nous avons toutefois relevé, lors de ce rechargement, des avertissements `UNEXPECTED/MISSING` sur les paramètres des tours audio et vision de Gemma — des composants multimodaux hérités du checkpoint de base, que notre fonction de déballage des couches `Gemma4ClippableLinear` convertit elle aussi au passage, car elle s'applique à tout le modèle plutôt qu'aux seules couches de langage. Sans conséquence pour nous, puisque notre tâche est purement textuelle et n'a jamais sollicité ces deux tours, mais un point à corriger (limiter le déballage aux couches du modèle de langage) si ce checkpoint devait un jour être réutilisé pour de l'audio ou de l'image.

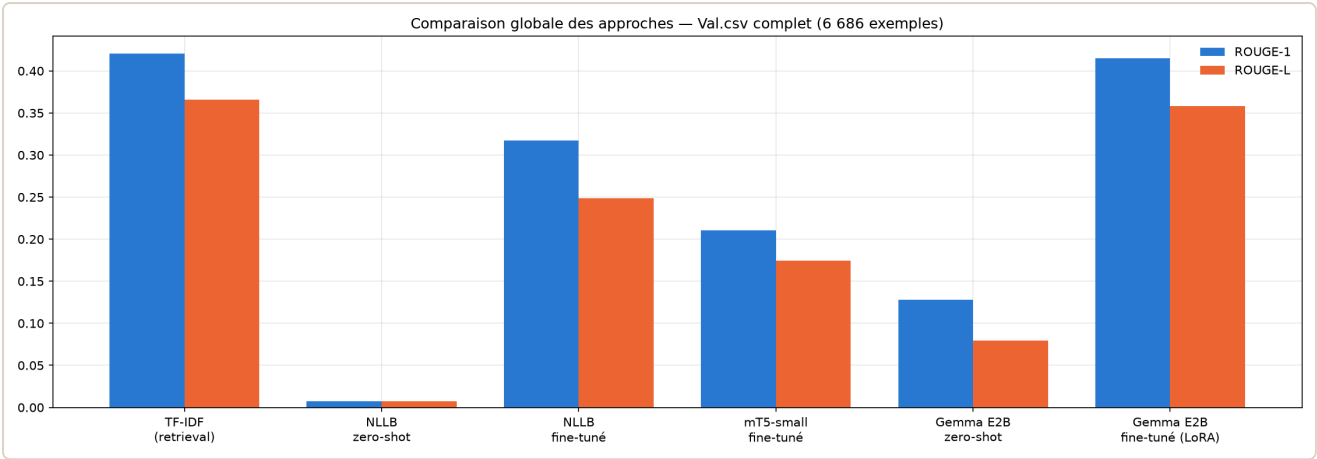
5.7 Démo en direct

En parallèle du travail sur les notebooks, nous avons construit une interface de test interactive (Gradio) pour manipuler directement le modèle fine-tuné : taper une question de santé dans n'importe laquelle des 5 langues et obtenir la réponse en streaming (mot par mot), avec une case à cocher pour comparer en parallèle avec la version avant fine-tuning. Nous avons retiré le sélecteur de langue de réponse initialement prévu : en Phase 2, la langue passée au prompt d'instruction était toujours celle de la question elle-même, jamais une langue cible différente — le modèle n'a donc jamais appris à traduire vers une langue distincte de celle de la question, et un sélecteur qui laisserait croire le contraire aurait été trompeur. Le modèle répond simplement dans la langue de la question posée.

par ailleurs de centraliser les checkpoints de mT5, TF-IDF et NLLB, développés séparément par les membres de l'équipe, sur un même environnement — un chantier que nous envisageons pour une itération future.

6. Comparaison des modèles

Les deux modèles sont désormais évalués sur le même jeu de validation complet (6 686 exemples), ce qui rend la comparaison chiffrée ci-dessous pleinement rigoureuse.



Comparaison ROUGE-1 / ROUGE-L de toutes les approches testées (Val.csv complet).

Critère	mT5-small	Gemma E2B (QLoRA)
Paramètres totaux	300 M	5,13 Md
Paramètres entraînés	300 M (100 %)	29,9 M (0,58 %)
Méthode d'adaptation	Full fine-tuning	QLoRA (4-bit + LoRA)
Durée d'entraînement	~2h33	~16h45
Overfitting observé	Non (sur la plage mesurée)	Oui (à partir du step ~6 400)
Taille échantillon éval.	6 686 (Val complet)	6 686 (Val complet)
ROUGE-1 (moyen)	0,2104	0,4149
ROUGE-L (moyen)	0,1745	0,3583

Nos deux modèles fine-tunés dépassent nettement les baselines de départ fournies par Zindi (TF-IDF 0,41/0,36 mis à part — voir la réserve méthodologique en section 2.1) : mT5 (0,21/0,17) et surtout Gemma fine-tuné (0,41/0,36) font mieux que le LLM fine-tuné du notebook de démarrage (0,32/0,25), ce qui nous conforte dans l'idée que notre travail d'exploration, de rééquilibrage des langues et de formatage des prompts (sections 3 à 5) a porté ses fruits par rapport à une adaptation plus sommaire du template de départ. Gemma dépasse mT5 sur les deux métriques, avec un écart substantiel (+0,20 ROUGE-1, +0,18 ROUGE-L) — un résultat cohérent avec sa taille (17 fois plus de paramètres), mais obtenu pour un coût d'entraînement environ 6,5 fois plus long et une adaptation de seulement 0,58 % des paramètres, ce qui nous semble être le compromis le plus intéressant des deux si les ressources de calcul le permettent.



Constat — En regardant uniquement les chiffres du graphique ci-dessus, le TF-IDF (0,42/0,37) devance même Gemma fine-tuné — mais nous ne le considérons pas comme le meilleur modèle du projet. La raison, déjà posée en section 2.1 : le TF-IDF ne *génère* aucune réponse, il recopie mot pour mot la question d'entraînement la plus proche. Ce chevauchement lexical parfait gonfle mécaniquement son ROUGE, un score qui mesure justement un chevauchement lexical — mais rien ne garantit que la réponse recopiée soit médicalement adaptée à une question inédite, contrairement aux modèles génératifs qui produisent une réponse pensée pour la question posée. Entre les approches réellement génératives (NLLB, mT5, Gemma), c'est donc **Gemma fine-tuné qui constitue en pratique le meilleur modèle de ce projet**, et de loin : +0,097 ROUGE-1 sur le meilleur des deux autres (NLLB fine-tuné, 0,3176).

7. Difficultés techniques rencontrées et résolues

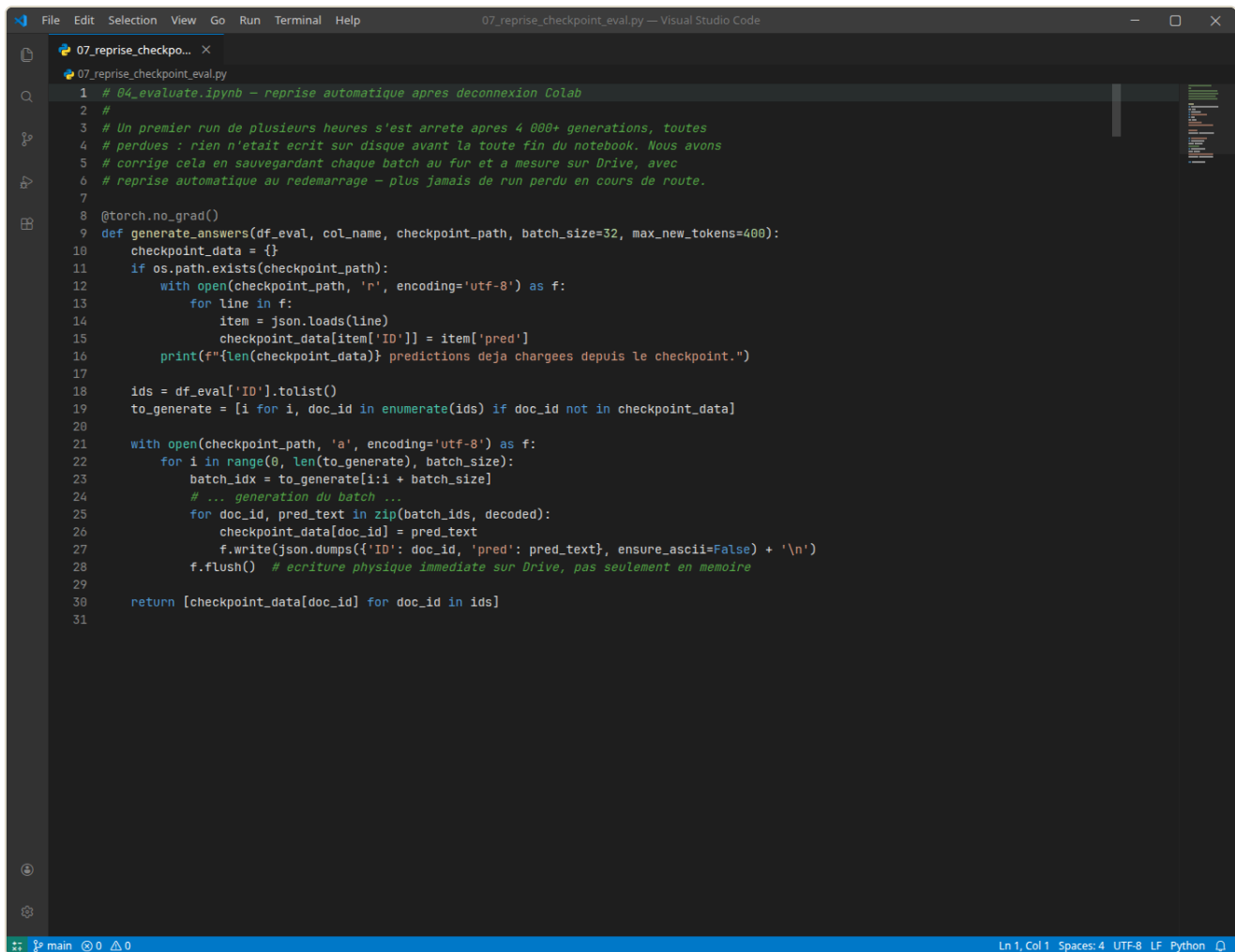
Nous documentons ici trois difficultés concrètes que nous avons rencontrées en cours de route, et la façon dont nous les avons résolues — elles nous semblent représentatives du travail d'ingénierie qu'a demandé ce projet, au-delà du seul entraînement des modèles.

7.1 Une classe non reconnue par PEFT

Les couches d'attention de ce checkpoint Gemma sont enveloppées dans une classe `Gemma4ClippableLinear` (clipping des activations) que la bibliothèque PEFT ne reconnaît pas pour y attacher un adaptateur LoRA. Nous avons résolu cela en débarrassant ces couches vers leur `Linear4bit` interne avant de configurer LoRA — ce qui désactive ce clipping pendant

l'entraînement. Nous avons surveillé ce risque de près : si la loss avait divergé ou produit des NaN, cela aurait signalé que ce clipping était nécessaire ; ce n'est pas ce que nous avons observé, la loss ayant décliné normalement sur tout l'entraînement.

7.2 Une évaluation de plusieurs heures perdue en une déconnexion



```
1 # 04_evaluate.ipynb - reprise automatique apres deconnexion Colab
2 #
3 # Un premier run de plusieurs heures s'est arrete apres 4 000+ generations, toutes
4 # perdues : rien n'etait ecrit sur disque avant la toute fin du notebook. Nous avons
5 # corrige cela en sauvegardant chaque batch au fur et a mesure sur Drive, avec
6 # reprise automatique au redemarrage - plus jamais de run perdu en cours de route.
7
8 @torch.no_grad()
9 def generate_answers(df_eval, col_name, checkpoint_path, batch_size=32, max_new_tokens=400):
10     checkpoint_data = {}
11     if os.path.exists(checkpoint_path):
12         with open(checkpoint_path, 'r', encoding='utf-8') as f:
13             for line in f:
14                 item = json.loads(line)
15                 checkpoint_data[item['ID']] = item['pred']
16             print(f'{len(checkpoint_data)} predictions deja chargees depuis le checkpoint.')
17
18     ids = df_eval[col_name].tolist()
19     to_generate = [i for i, doc_id in enumerate(ids) if doc_id not in checkpoint_data]
20
21     with open(checkpoint_path, 'a', encoding='utf-8') as f:
22         for i in range(0, len(to_generate), batch_size):
23             batch_idx = to_generate[i:i + batch_size]
24             # ... generation du batch ...
25             for doc_id, pred_text in zip(batch_idx, decoded):
26                 checkpoint_data[doc_id] = pred_text
27                 f.write(json.dumps({'ID': doc_id, 'pred': pred_text}, ensure_ascii=False) + '\n')
28             f.flush() # ecriture physique immediate sur Drive, pas seulement en memoire
29
30     return [checkpoint_data[doc_id] for doc_id in ids]
```

Extrait — reprise automatique après déconnexion Colab.

Notre premier run d'évaluation sur l'ensemble de `Val.csv` s'est arrêté après plus de 4 000 générations, à la suite d'une déconnexion de la session Colab — et nous avons perdu l'intégralité de cette progression. La cause : notre fonction de génération accumulait toutes les réponses en mémoire et n'écrivait rien sur disque avant la toute fin du notebook. Nous avons corrigé cela en sauvegardant chaque batch au fur et à mesure sur Drive, avec un mécanisme de reprise qui relit les prédictions déjà générées et ne recalcule que ce qui manque — un run interrompu ne nous coûte plus désormais qu'un batch, jamais des heures de calcul.

7.3 Une progression de GPU dictée par la mémoire disponible

Nous avons changé de GPU à trois reprises au cours du projet, chaque fois pour une raison précise : le T4 initial (16 Go) pour le setup et l'exploration ; le L4 (24 Go, bf16 natif) pour l'entraînement QLoRA, après avoir constaté qu'un batch de 8 provoquait un OOM sur le calcul de la perte (logits sur tout le vocabulaire) ; l'A100 (80 Go) pour la fusion de l'adaptateur en pleine précision (section 5.6), où même le L4 s'est révélé insuffisant pour charger le modèle complet en bf16 non quantifié.

8. Conclusion et recommandations

Ce projet nous a permis de mettre en œuvre et de comparer deux approches de fine-tuning de modèles de langage pour la tâche de questions-réponses médicales multilingues du challenge Zindi : un full fine-tuning d'un modèle compact (mT5-small) et une adaptation QLoRA d'un modèle bien plus volumineux (Gemma E2B). Partis du notebook de démarrage fourni par l'organisateur, dont le LLM baseline fine-tuné plafonnait à 0,32/0,25 de ROUGE — en dessous même du simple retrieval TF-IDF — nous avons développé deux pipelines complets allant nettement plus loin. Les deux pipelines partagent une même base de données, explorée et diagnostiquée en commun, ce qui nous a permis d'identifier dès la phase exploratoire des signaux (déséquilibre des langues, hétérogénéité de la tokenisation) que nous avons retrouvés, de façon cohérente, dans les résultats d'évaluation des deux modèles — et même dans ceux des baselines Zindi. Nous avons par ailleurs étendu l'évaluation de Gemma à l'ensemble du jeu de validation (section 5.4), ce qui nous a permis d'aboutir à une comparaison chiffrée pleinement rigoureuse entre les deux modèles (section 6) : Gemma fine-tuné (0,41/0,36) devance mT5 (0,21/0,17) sur les deux métriques.

Recommandations pour une itération future

- **Cibler l'amharique et le luganda en priorité** : sur-échantillonnage renforcé, données supplémentaires, voire vérification/adaptation du tokeniseur pour mieux supporter le script Ge'ez (amharique) ;
- **Introduire un arrêt anticipé (early stopping)** pour Gemma, afin d'éviter les ~5-6 heures de calcul supplémentaires qui ont surtout produit du sur-apprentissage sans bénéfice sur ce run ;
- **Compléter l'évaluation automatique (ROUGE) par une évaluation humaine ou par un LLM juge**, le score ROUGE ne mesurant qu'un chevauchement lexical et non la fiabilité médicale réelle des réponses générées ;

- **Pour la mise en production**, nous privilégierions le modèle offrant le meilleur compromis qualité / coût de calcul selon le contexte — Gemma fusionné en modèle autonome pour une meilleure qualité, ou mT5 pour un déploiement plus léger et rapide à entraîner.

Projet réalisé par Andil Mchangama, Nasma Youssouf M'madi, Abdoulaye Diouf, Harouna Mbaye et Hamidou Mbodji — Master 2 Business Intelligence, Université Cheikh Anta Diop de Dakar — Cours de NLP.